Python for Artificial Intelligence

Neural Networks in PyTorch

Week 5, Day 1

Today's Project: VillainNet

- Yesterday you hand-scored one ride with weights you picked. Today the network learns its own weights, for real, and we switch to a more fun dataset from Kaggle: Marvel and DC comic characters
- The question: can a neural network tell a hero from a villain just from a character's description (publisher, gender, eye and hair colour, secret identity, how often they appear)? We call it VillainNet
- Hour 1: load and prepare the character data from Kaggle, then build VillainNet the hard way: plain NumPy, no library, every formula by hand
- Hour 2: build the same network the easier way in TensorFlow (Keras), then start on PyTorch: tensors and defining the network
- Hour 3: PyTorch's training loop, compare all three builds with classical models, then test the network on famous characters and ask what it really learned

Hour 1

Data and the Hard Way: No Library

PROJECT

Meet the Comic Characters

- A dataset from FiveThirtyEight, on Kaggle: 23,272 Marvel and DC characters taken from the fan wikis of both publishers. Each row is one character
- Columns include publisher, sex, identity (secret or public), eye colour, hair colour, alive or deceased, number of appearances, year of first appearance, and ALIGN: good, bad or neutral
- Our label is ALIGN. We keep the good and bad characters and drop neutral and unlabelled ones, which leaves 17,083 rows: 9,615 villains and 7,468 heroes
- Because we are working in Kaggle: open your notebook, click Add Input, search for "FiveThirtyEight comic characters", and add the dataset. The files then appear under /kaggle/input/, and you read them with pd.read_csv like any other CSV

EXAMPLE

Loading the Comic Data in Kaggle

Two plain pd.read_csv calls. In Kaggle, click a file in the Input panel on the right and use Copy file path if your folder name differs.

```
import numpy as np, pandas as pd

folder = '/kaggle/input/fivethirtyeight-comic-characters-dataset/'     # your path may differ
dc = pd.read_csv(folder + 'dc-wikia-data.csv')
marvel = pd.read_csv(folder + 'marvel-wikia-data.csv')
for d in (dc, marvel):
    d.columns = d.columns.str.upper()                    # Marvel writes Year, DC writes YEAR
dc['PUBLISHER'], marvel['PUBLISHER'] = 'DC', 'Marvel'

df = pd.concat([dc, marvel], ignore_index=True)
print(df.shape)                                      # (23272, 14)

df = df[df['ALIGN'].isin(['Good Characters', 'Bad Characters'])]
print(df.shape)                                      # (17083, 14)
print(df['ALIGN'].value_counts().to_dict())
# {'Bad Characters': 9615, 'Good Characters': 7468}
```

PROJECT

Preparing the Characters for a Neural Network

- Networks work with numbers in arrays, not DataFrames with text columns, so the data needs preparing first
- Text columns such as eye colour become one-hot columns. Rare values (a colour seen only a few times) are grouped as Other, and missing values become their own Unknown column, since a missing eye colour often means a minor character
- Two numeric columns are added: the number of appearances (log scaled, because a few characters appear thousands of times) and the year of first appearance
- The label is 1 for a villain and 0 for a hero. The classes are not equal: 56.3 percent are villains, so a model that always says villain is right 54 percent of the time on our test set. That is the score to beat

EXAMPLE

Building the Features

One helper keeps the most common values of a column and groups the rest as Other, then get_dummies does the one-hot encoding.

```
def tidy(col, keep):
    s = df[col].fillna('Unknown').str.replace(' Characters| Eyes| Hair| Identity', '',
regex=True)
    return s.where(s.isin(s.value_counts().index[:keep]), 'Other')

X = pd.get_dummies(pd.DataFrame({
    'publisher': df['PUBLISHER'], 'sex': tidy('SEX', 2), 'identity': tidy('ID', 3),
    'alive': tidy('ALIVE', 2), 'eye': tidy('EYE', 5), 'hair': tidy('HAIR', 5)}), dtype=float)
X['appearances'] = np.log1p(df['APPEARANCES'].fillna(1))
X['year'] = df['YEAR'].fillna(df['YEAR'].median())
y = (df['ALIGN'] == 'Bad Characters').astype(int)      # 1 = villain

print(X.shape, round(y.mean(), 3))                      # (17083, 26) 0.563
print(list(X.columns[:6]))
# ['publisher_DC', 'publisher_Marvel', 'sex_Female', 'sex_Male', 'sex_Other', 'identity_Other']
```

EXAMPLE

Scaling the Character Data

Split 80/20 into train and test, keep a fifth of the training data for validation, then StandardScaler (fit on the training data only). The result is plain NumPy arrays, used by all three builds today.

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=42)
test_names = df.loc[X_test.index, 'NAME'].values     # remember who is who

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_val, X_test = scaler.transform(X_val), scaler.transform(X_test)
y_train, y_val, y_test = [v.values.reshape(-1, 1) for v in (y_train, y_val, y_test)]
print(X_train.shape, X_val.shape, X_test.shape)    # (10932, 26) (2734, 26) (3417, 26)
```

CONCEPT

Three Ways to Build a Network

The same VillainNet can be built with no library at all, with TensorFlow (Keras), or with PyTorch. Today we go from the hardest way to the easier ones.

	No library (NumPy)	TensorFlow (Keras)	PyTorch
Order today	1. The hard way	2. Easier	3. Easiest to see inside
Gradients	You derive and code every one by hand	Automatic (GradientTape)	Automatic (autograd)
GPU	No	Yes	Yes
Layers and optimizers	You write them	Dense, built-in optimizers	nn.Linear, torch.optim
Training loop	Written by hand	One call: model.fit()	Written by hand (4 steps)
Best for	Learning exactly what happens	Quick builds and deployment	Research and flexible experiments

EXAMPLE

No Library: Setup and Forward Pass

VillainNet in plain NumPy. X_train holds the 26 scaled character columns and y_train the 0/1 labels (1 = villain), prepared on the previous slides.

```
import numpy as np

rng = np.random.default_rng(42)
W1 = rng.normal(0, 0.5, (26, 8));  b1 = np.zeros(8)    # hidden layer
W2 = rng.normal(0, 0.5, (8, 1));  b2 = np.zeros(1)    # output layer
sigmoid = lambda z: 1 / (1 + np.exp(-z))

# forward pass
z1 = X_train @ W1 + b1
a1 = np.maximum(0, z1)                  # ReLU
p  = sigmoid(a1 @ W2 + b2)              # probability of villain

# loss: mean squared error
loss = np.mean((p - y_train) ** 2)
```

EXAMPLE

No Library: Backward Pass and Update

Backpropagation written out by hand: the chain rule, layer by layer, then one gradient descent step.

```
# backward pass (chain rule)
dz2 = 2 * (p - y_train) * p * (1 - p) / len(y_train)   # error at the output (MSE, then sigmoid)
dW2 = a1.T @ dz2;  db2 = dz2.sum(axis=0)
dz1 = (dz2 @ W2.T) * (z1 > 0)             # ReLU passes gradient only where z1 > 0
dW1 = X_train.T @ dz1;  db1 = dz1.sum(axis=0)

# update: gradient descent
lr = 0.5
W1 -= lr * dW1;  b1 -= lr * db1
W2 -= lr * dW2;  b2 -= lr * db2

# put forward pass + loss + backward pass + update in a loop of 500 epochs
```

CONCEPT

What a Framework Does for You

Compare the by-hand version with what a framework (Keras or PyTorch) does for the same network.

Job	By hand (NumPy)	With a framework (Keras or PyTorch)
Weights and biases	Create W1, b1, W2, b2 yourself	Dense / nn.Linear creates and tracks them
Gradients	Write every formula (dz2, dW2, dz1, dW1...)	Automatic: GradientTape or loss.backward()
Weight updates	W -= lr * dW for every parameter	The optimizer does it for you
Changing the network	Re-derive the gradients	Change the layer definition
Risk of mistakes	High: one wrong formula breaks training silently	Low: autograd is thoroughly tested

Hour 2

The Easier Way: TensorFlow, then PyTorch Basics

EXAMPLE

VillainNet in TensorFlow (Keras)

The same 26-8-1 network, now with Keras. It hides the loop and the calculus inside compile() and fit().

```
import tensorflow as tf

tf.keras.utils.set_random_seed(42)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(26,)),
    tf.keras.layers.Dense(8, activation="relu"),       # hidden layer
    tf.keras.layers.Dense(1, activation="sigmoid"),    # output layer
])
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
              loss="mse", metrics=["accuracy"])

model.fit(X_train, y_train, validation_data=(X_val, y_val),
          epochs=100, batch_size=len(X_train))
loss, acc = model.evaluate(X_test, y_test)      # test accuracy: 0.670
```

CONCEPT

Keras: What Each Line Does

Compare with the NumPy version: no gradients, no update lines, no loop.

Line	What it does
Sequential([...])	Stacks layers in order, like the forward pass you wrote by hand
Dense(8, activation="relu")	Hidden layer: 26 inputs to 8 neurons with ReLU (W1, b1 created for you)
Dense(1, activation="sigmoid")	Output layer: 8 values to one villain probability
compile(optimizer, loss)	Picks Adam and mean squared error, the loss you coded by hand
fit(...)	Forward pass, loss, gradients and updates for every epoch, in one call
evaluate(X_test, y_test)	Scores the trained network on the 3,417 unseen test characters

CONCEPT

What Is PyTorch?

- PyTorch is the most widely used framework for building and training neural networks in Python, and it is the one most modern AI research is written in
- It handles two things you don't want to write by hand: computing gradients automatically, and running computations fast (optionally on a GPU)
- Its core data structure is the tensor, which behaves almost exactly like a NumPy array
- If you can use NumPy, you already know most of what you need to start using PyTorch

EXAMPLE

Tensors: NumPy's Cousin

A tensor is created and used almost identically to a NumPy array.

```
import torch

x = torch.tensor([0.5, 0.2, 0.9])    # one character's scaled features
w = torch.tensor([0.4, -0.3, 0.8])   # weights

z = torch.dot(x, w) + 0.1
print(z)              # tensor(0.9600)
print(z.shape)        # torch.Size([])
```

EXAMPLE

From NumPy Arrays to Tensors

The arrays prepared earlier become float32 tensors, the data type PyTorch networks expect.

```
import torch

to_t = lambda a: torch.tensor(a, dtype=torch.float32)
X_train, X_val, X_test = to_t(X_train), to_t(X_val), to_t(X_test)
y_train, y_val, y_test = to_t(y_train), to_t(y_val), to_t(y_test)

print(X_train.shape)    # torch.Size([10932, 26])
print(y_train.shape)    # torch.Size([10932, 1])
```

CONCEPT

Why Not Just Use NumPy?

- Tensors can run on a GPU, where the same matrix operations happen far faster, essential for large networks
- Tensors support autograd: PyTorch automatically tracks every operation and can compute gradients for you
- That second point replaces backpropagation's math entirely, you call one method, PyTorch does the calculus
- For today's small network, speed won't matter, but the autograd habit you build today scales to everything later

EXAMPLE

Defining VillainNet

nn.Module is the standard way to define a network's layers and how data flows through them.

```
import torch.nn as nn

class VillainNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(26, 8)  # 26 character columns -> 8 hidden neurons
        self.output = nn.Linear(8, 1)   # 8 hidden -> 1 output (villain?)

    def forward(self, x):
        x = torch.relu(self.hidden(x))
        x = torch.sigmoid(self.output(x))
        return x

torch.manual_seed(42)               # same starting weights every run
model = VillainNet()
```

CONCEPT

Reading That Network

- nn.Linear(26, 8) is one layer: it takes 26 inputs (publisher, sex, identity, eye and hair colour, alive or deceased, appearances and year) and produces 8 numbers out, the weighted-sum step from Day 19
- torch.relu and torch.sigmoid are the activation functions from yesterday, applied right after each layer
- forward() describes the forward pass: how a character's features flow through the layers to produce a villain probability
- This is the exact same anatomy as yesterday's hand-scored neuron, now written as runnable code

CONCEPT

VillainNet, Layer by Layer

What model.parameters() counts: every weight and bias the optimizer will adjust.

Layer	Shape	Parameters	What it does
Input	26 columns	0	One character's 26 scaled features
hidden: nn.Linear(26, 8)	26 in, 8 out	26 x 8 + 8 = 216	Weighted sums for 8 hidden neurons
ReLU	8 values	0	Keeps positives, zeroes out negatives
output: nn.Linear(8, 1)	8 in, 1 out	8 x 1 + 1 = 9	One weighted sum of the hidden values
Sigmoid	1 value	0	Turns that sum into a villain probability
Total		225	Weights and biases the optimizer learns

QUICK CHECK

Quick Check

In VillainNet above, why is the final activation sigmoid rather than ReLU?

ANSWER

Quick Check: Answer

```
# The task is binary classification (hero or villain), so the
# output needs to be a probability between 0 and 1.
#
# Sigmoid squashes any number into that 0-to-1 range, exactly
# like Logistic Regression's output on Day 18. ReLU only
# clips negative numbers to zero, it doesn't bound the top,
# so it isn't suitable as a final layer for a probability.
```

15-Minute Break

Stretch, grab a coffee. We'll resume shortly.

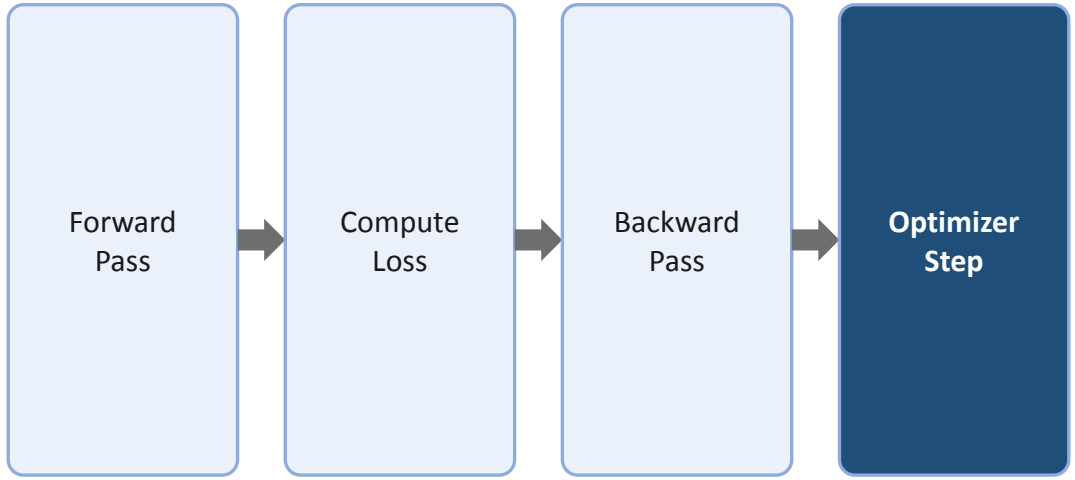
Hour 3

PyTorch Training, Then Comparing All Three

CONCEPT

Four Steps, Repeated Every Batch

- Every PyTorch training loop repeats the same four steps, for every batch, every epoch
- Forward pass: feed character features through the model to get a villain probability
- Compute loss: compare that probability to what actually happened with a loss function
- Backward pass & update: compute gradients automatically, then nudge every weight to reduce the loss



EXAMPLE

Loss Function & Optimizer

The loss function matches the task; the optimizer is the algorithm that performs gradient descent.

```
loss_fn = nn.MSELoss()                    # mean squared error, the same loss as Day 17
regression
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
# Adam is a smarter, adaptive version of plain gradient descent from Day 19
```

EXAMPLE

The Training Loop, Written Out

This is the forward/loss/backward/update cycle from the diagram, now as real code.

```
num_epochs = 100

for epoch in range(num_epochs):
    predictions = model(X_train)          # forward pass
    loss = loss_fn(predictions, y_train)  # compute loss

    optimizer.zero_grad()                 # clear old gradients
    loss.backward()                       # backward pass (autograd!)
    optimizer.step()                      # update every weight
```

CONCEPT

The Training Loop, Line by Line

Every line maps to an idea from Day 19.

Code	Day 19 idea	What it does
predictions = model(X_train)	Forward propagation	Pushes every training character through the network
loss = loss_fn(predictions, y_train)	Loss calculation	Scores how wrong the probabilities are
optimizer.zero_grad()	(housekeeping)	Clears the gradients left from the last epoch
loss.backward()	Backpropagation	Works out each weight's gradient with the chain rule
optimizer.step()	Gradient descent (Adam)	Moves every weight a small step downhill

CONCEPT

loss.backward(): Autograd in Action

- loss.backward() is backpropagation from Day 19, computed automatically for every weight in VillainNet
- optimizer.zero_grad() is easy to forget, gradients accumulate by default, so clear them before every new batch
- optimizer.step() is the actual gradient descent update, moving every weight slightly downhill
- These three lines, in this order, appear in almost every PyTorch training script you will ever write

QUICK CHECK

Quick Check

What would likely go wrong if you forgot to call optimizer.zero_grad() before each loss.backward()?

ANSWER

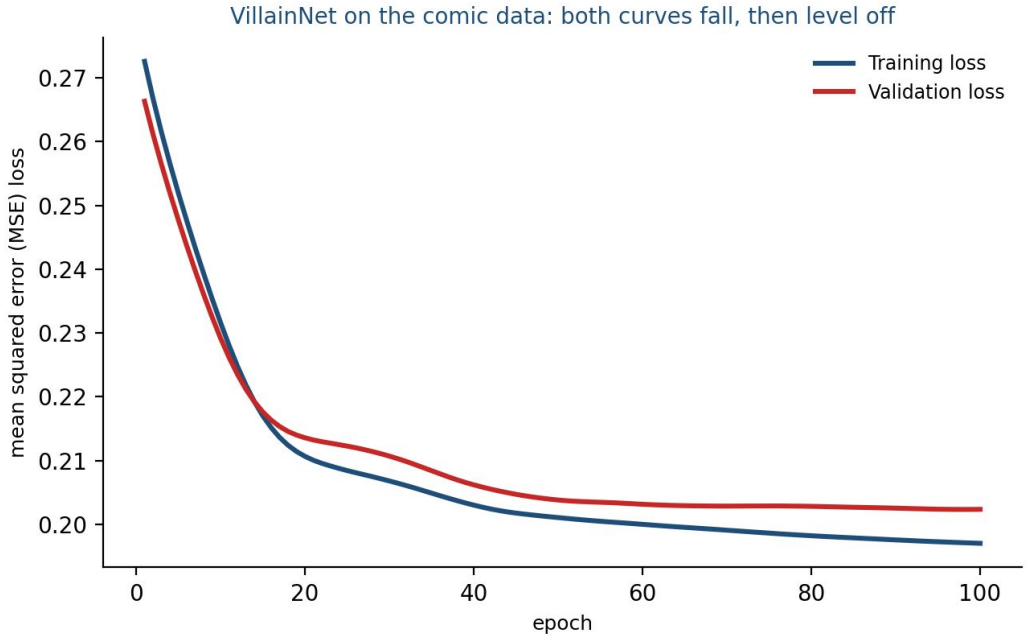
Quick Check: Answer

```
# Gradients from previous batches would accumulate (add up)
# instead of being replaced each time.
#
# The weight updates would end up based on a sum of many
# batches' gradients rather than just the current one, which
# usually makes training unstable or causes it to converge
# to the wrong place.
```

CONCEPT

Reading Your Own Loss Curve

- After each epoch, compute and record the loss on both the training set and the validation set
- A healthy run looks like this: both curves drop together and then flatten out at about the same level
- Notice that the loss stops near 0.20, not near 0: a perfect model would reach 0, but this task is genuinely hard, as the results will show
- Compare with Day 19's warning curve, where validation loss climbed while training loss kept falling. If you see that gap open up, simplify the model, add data, or stop earlier



EXAMPLE

Evaluating Without Training

torch.no_grad() turns off autograd tracking, since no training happens during evaluation.

```
model.eval()
with torch.no_grad():
    probs = model(X_test)
    predicted_classes = (probs > 0.5).float()
    accuracy = (predicted_classes == y_test).float().mean()
    print(f'Test accuracy: {accuracy:.3f}')      # 0.668

from sklearn.metrics import classification_report
print(classification_report(y_test.numpy(), predicted_classes.numpy()))
```

COMPARISON

PyTorch vs. TensorFlow

Both are mature and widely used. Keras is TensorFlow's high-level interface.

	TensorFlow (Keras)	PyTorch
Created by	Google	Meta (originally Facebook AI Research)
Training style	model.fit() runs the loop for you	You write the loop: forward, loss, backward, step
Feel	More built-in structure, less code for standard models	Plain Python, easy to inspect and debug step by step
Strong points	Mature tools for deployment, such as mobile and browser versions	Very popular in research and for modern language models
Same underneath	Tensors, automatic gradients, GPU support	Tensors, automatic gradients, GPU support

RESULTS

Same Network, Three Ways

VillainNet (26 inputs, 8 hidden, 1 output, 225 parameters) built three ways and scored on the same 3,417 test characters, in the order we built them.

Built with	Optimizer and length	Test accuracy	F1
No library (NumPy)	Gradient descent, lr 0.5, 500 epochs	0.67	0.71
TensorFlow (Keras)	Adam, lr 0.01, 100 epochs	0.67	0.72
PyTorch	Adam, lr 0.01, 100 epochs	0.67	0.72

CONCEPT

Why the Numbers Differ

- The network is identical, so the library is not what changes the result
- Random starting weights, the optimizer and the number of epochs do: each run begins from a different point and can end in a slightly different place
- With 3,417 test characters, differences of a point or two are small. All three builds land within one point of each other (accuracy 0.67, F1 0.71 to 0.72)
- The honest conclusion: the library does not matter here. What matters is how much the data can tell us, as the next slides show

REFERENCE

Classical Models vs. VillainNet

On a small, structured dataset like the comic table, a tiny neural network rarely beats the classical models.

Model	Typical Fit	When It's the Better Choice
Logistic Regression / Trees	Fast, simple, interpretable	Small, structured/tabular data, most real business problems
VillainNet (small NN)	More setup, no real accuracy gain here	Mainly useful as a learning step toward larger networks
Large Neural Network	Needs lots of data and compute	Images, text, audio, and other unstructured, high-volume data

RESULTS

VillainNet vs. Classical Models

Four classical models trained on the same split and scored on the same 3,417 test characters (class 1 = villain). The first row is the score to beat.

Model	Accuracy	Precision	Recall	F1
Always guess "villain"	0.54	0.54	1.00	0.70
Logistic Regression	0.66	0.66	0.79	0.72
KNN (K=5)	0.65	0.66	0.74	0.70
Decision Tree (depth 4)	0.64	0.64	0.77	0.70
Random Forest (100 trees)	0.67	0.68	0.73	0.70
VillainNet (PyTorch, 100 epochs)	0.67	0.67	0.78	0.72

CONCEPT

What the Comparison Tells Us

- VillainNet (PyTorch) scores 0.67 accuracy, 13 points above the 0.54 you get by always guessing villain. It learned something real
- But it only equals the classical models: Random Forest also gets 0.67, Logistic Regression 0.66. Every model, simple or complex, stops at about the same level
- When a simple and a complex model hit the same ceiling, the limit is usually in the data, not in the model. A character's looks and publisher say only so much about whether the writers made them good or evil
- The comic table is also small and structured, which is where classical models are hard to beat
- Neural networks pay off with large amounts of unstructured data such as text, images and audio, which is where this course heads next

QUICK CHECK

Guess First

VillainNet gives each character a villain probability. Before we look, guess which of these five it gets WRONG, using only what it sees:

Batman: male, secret identity, blue eyes, black hair, 3,093 appearances
Joker: male, secret identity, green eyes, green hair, 517 appearances
Wonder Woman: female, public identity, blue eyes, black hair, 1,231 appearances
Jean Grey: female, public identity, green eyes, red hair, deceased, 1,107 appearances
Alexander Luthor: male, public identity, green eyes, no hair listed, 677 appearances

EXAMPLE

Famous Characters: What the Network Said

These five characters were in the test set, which the network never trained on. Probability of villain from the PyTorch model.

```
probs = model(X_test).detach().numpy().ravel()
names = ['Batman (Bruce Wayne)', 'Joker (New Earth)', 'Wonder Woman (Diana Prince)',
         'Jean Grey (Earth-616)', 'Alexander Luthor (New Earth)']
for name in names:
    i = list(test_names).index(name)
    truth = 'villain' if y_test[i] == 1 else 'hero'
    print(f'{name:30} P(villain) = {probs[i]:.2f}   truth: {truth}')
# Batman (Bruce Wayne)           P(villain) = 0.04   truth: hero
# Joker (New Earth)              P(villain) = 0.13   truth: villain
# Wonder Woman (Diana Prince)    P(villain) = 0.01   truth: hero
# Jean Grey (Earth-616)          P(villain) = 0.07   truth: hero
# Alexander Luthor (New Earth)   P(villain) = 0.11   truth: villain
# It gets the Joker and Alexander Luthor WRONG, with high confidence.
```

CONCEPT

What Did It Actually Learn?

- The network never read a story. It only saw columns. Famous heroes such as Batman and Wonder Woman have thousands of appearances, and in this data a high appearance count points strongly to "hero"
- Logistic Regression on the same 26 columns gives a readable view (negative pushes towards hero, positive towards villain): appearances -0.47, public identity -0.24, female -0.21 on the hero side, and secret identity +0.22, male +0.15 on the villain side
- The Joker has a secret identity, is male and has fewer appearances than the big heroes, yet the network still says hero. Rare, flamboyant villains are exactly where a table of columns fails
- So VillainNet is not judging morality. It is picking up how the writers of these comics have tended to build their characters

CONCEPT

A Warning About Bias

- The readable view on the previous slide leans towards "hero" for female characters (and the network is fed the same column). That is not a fact about women. It reflects who the writers chose to make villains in these comics
- A model trained on biased data repeats the bias, and can look accurate while doing it. Accuracy of 0.67 says nothing about whether the pattern is fair or true
- Never use a model like this to judge real people. The same mistake, using appearance or background columns to predict a person's character, is made in real hiring and lending systems
- Good habits: ask what the labels really mean, check which columns drive the prediction, and compare results across groups before trusting a model

CONCEPT

A Few Habits Worth Keeping

- Always scale your inputs, unscaled features can make training unstable or painfully slow
- model.train() and model.eval() switch certain layers' behavior between training and evaluation, call them before each phase
- torch.save(model.state_dict(), 'villainnet.pt') saves a trained model so you don't have to retrain it every time
- Everything here, tensors, the training loop, loss curves, scales up directly to the much larger networks behind modern AI

PRACTICE

Practice

Train VillainNet from today for 100 epochs on the comic data, recording training and validation loss each epoch. Plot both curves. Then train Logistic Regression, KNN, a Decision Tree and a Random Forest on the same split and compare their test results with VillainNet's and with the 0.54 baseline. Finally, remove the appearances column and retrain: how much does the accuracy change, and what does that tell you about what the network learned?

Key Takeaways & Week 4 Wrap-Up

- A PyTorch tensor behaves like a NumPy array, with automatic gradients and optional GPU speed added on top
- Every training loop repeats the same four steps: forward pass, compute loss, backward pass, optimizer step
- The same network can be built with no library, in TensorFlow (Keras), or in PyTorch: frameworks save you the calculus and the bookkeeping, not the thinking
- loss.backward() performs Day 19's backpropagation for you automatically, no calculus required
- This week you built a fare estimator and a quick-pickup classifier on the ride data, then trained a neural network three ways on a fresh Kaggle dataset: Marvel and DC characters. It beat the baseline but not the classical models, and it showed how a model can learn the habits of a dataset instead of the truth
- Coming up next: moving from structured, tabular data toward the text-based models at the heart of this course